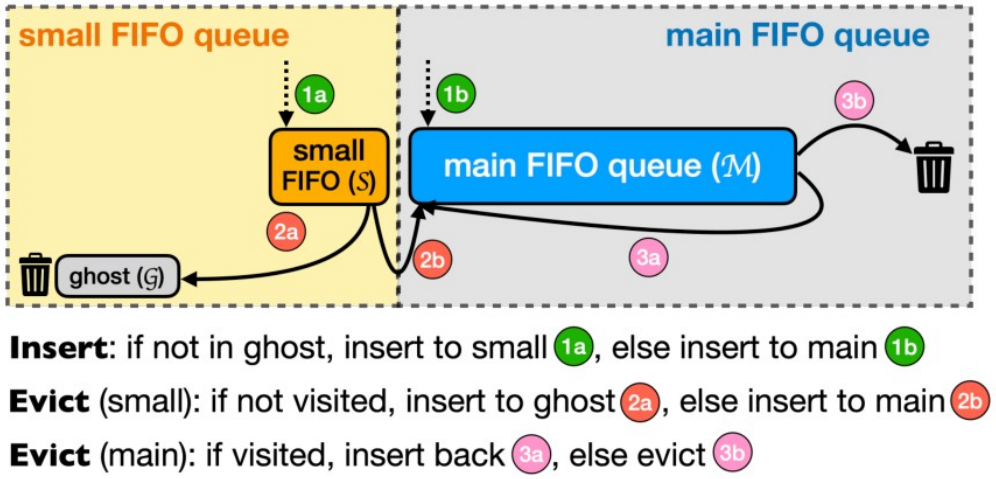
OPERATING SYSTEMS · PAPER PRESENTATION

S3-FIFO: Cache Eviction

FIFO Queues Are All You Need for Cache Eviction



Original Figure 5 · S3-FIFO architecture

Source: Yang et al., SOSP 2023

Three members · 10 minutes

SOSP 2023 · Juncheng Yang et al.

Team Structure & Timing

Three speakers follow the supplied script: background, algorithm, then evaluation.

Speaker	Time	Main responsibility
Member 1	3 min	Background, cache eviction, FIFO / LRU, one-hit wonders.
Member 2	4 min	Three FIFO queues, cache requests and eviction rules.
Member 3	3 min	Experimental setup, results, limitations and conclusion.

10-minute theoretical presentation · Results are reported by the original paper's authors.

PART 1 · MEMBER 1

Background & Motivation

Why does a cache need a better eviction policy?

Background → FIFO vs LRU → One-hit Wonders

What Is Cache Eviction?

Cache space is fast but limited; the system must decide what stays and what goes.

Small & fast cache

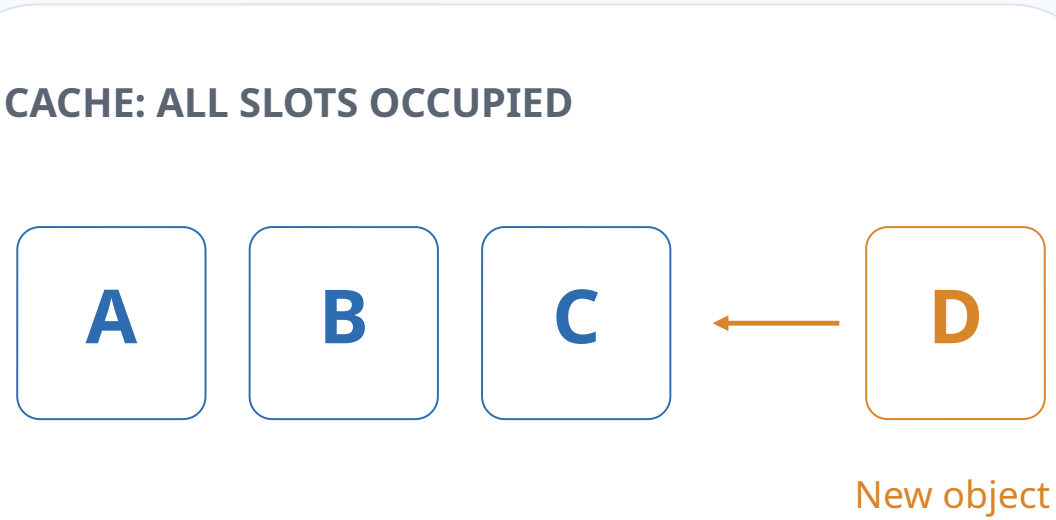
A cache is a small and fast storage space that keeps data we may need again.

Limited space

When the cache is full, the system must decide which data to remove.

Cache eviction

This process is called cache eviction.



Key question: what should stay in the cache, and what should go?

Source: Yang et al., SOSP 2023, §§1–2.

FIFO vs LRU

Both choose eviction candidates, but their update costs are different.

FIFO

First In, First Out

- It removes the data that entered the cache first.
- FIFO is simple and fast, but it may remove useful data.

LRU

Least Recently Used

- It removes data that has not been used for the longest time.
- LRU updates object order on access.
- This can slow the system down when many threads work together.

The goal: combine FIFO's simplicity with better cache efficiency.

Source: Yang et al., SOSP 2023, §2.2.

One-Hit Wonders & Quick Demotion

Shorter access sequences contain a higher fraction of objects with no reuse.

One-hit wonders

Many objects are used only once before they leave the cache: one-hit wonders.

Wasted cache space

They take up cache space without being used again.

Quick demotion

Quick demotion removes less useful objects early.

A SHORTER SEQUENCE → A HIGHER RATIO



start time	end time	sequence length (# objects)	# one-hit wonder	one-hit-wonder ratio
1	17	5	1 (E)	20%
1	7	4	2 (C, D)	50%
1	4	3	2 (B, C)	67%

Original Figure 1 · Source: Yang et al., SOSP 2023

Can we combine the simplicity of FIFO with better cache efficiency?

Source: original Figure 1 and §3, Yang et al., SOSP 2023.

PART 2 · MEMBER 2

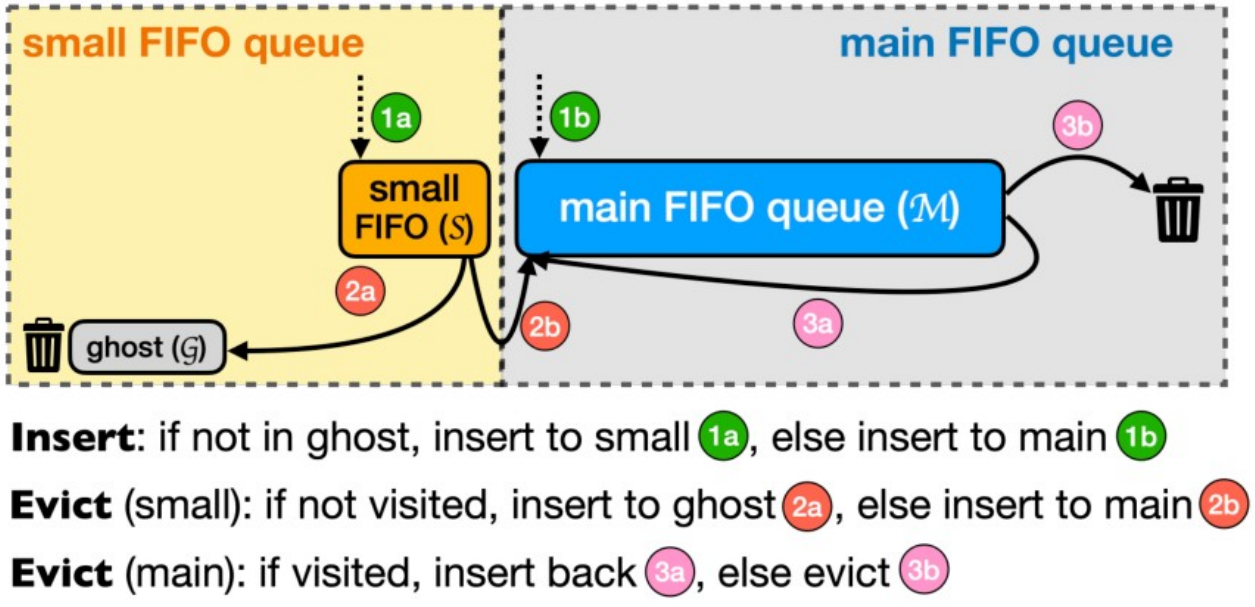
S3-FIFO Design & Algorithm

How do three simple FIFO queues work together?

Small / Main / Ghost → Requests → Eviction

Three FIFO Queues, One Cache

S3-FIFO uses Small, Main and Ghost queues instead of a single cache queue.



Original Figure 5 · Source: Yang et al., SOSP 2023

S **Small**

Filter new objects

M **Main**

Keep useful objects

G **Ghost**

Remember removed IDs

The main idea: use three FIFO queues to filter, protect and remember.

Source: Figure 5 and §4.1, Yang et al., SOSP 2023.

Small & Main: Two Data Queues

Small filters new objects; Main retains objects that have demonstrated reuse.



SMALL QUEUE

10%

New objects usually enter S first.

S is a testing area: objects with no reuse can leave quickly.

MAIN QUEUE

90%

M keeps objects that have shown they are useful.

Useful objects stay in the cache for longer.

Small + Main account for the actual cached data capacity.

Source: §4.1, Yang et al., SOSP 2023.

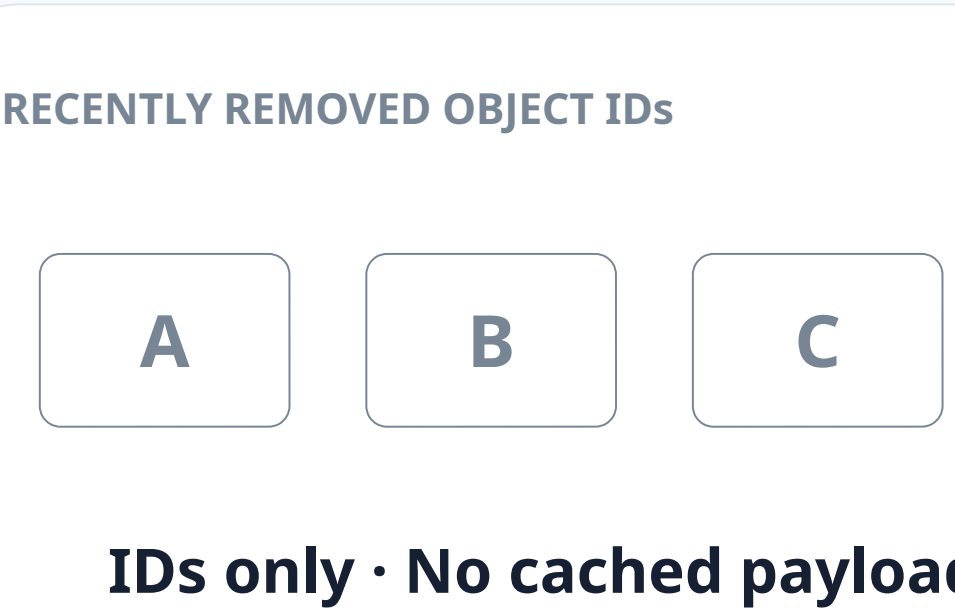
Ghost Queue: Remember IDs, Not Data

Ghost history helps recognize recently removed objects that may be useful again.

G remembers the IDs of recently removed objects.

It does not store the actual data.

The history can affect where an object is admitted next.



A remembered ID is useful history, but it is not a cache hit.

Source: §4.1, Yang et al., SOSP 2023. Diagram is a schematic, not experimental data.

Cache Hit: Update the Counter

A request is a hit when the object is already in Small or Main.



The access counter is capped at three.

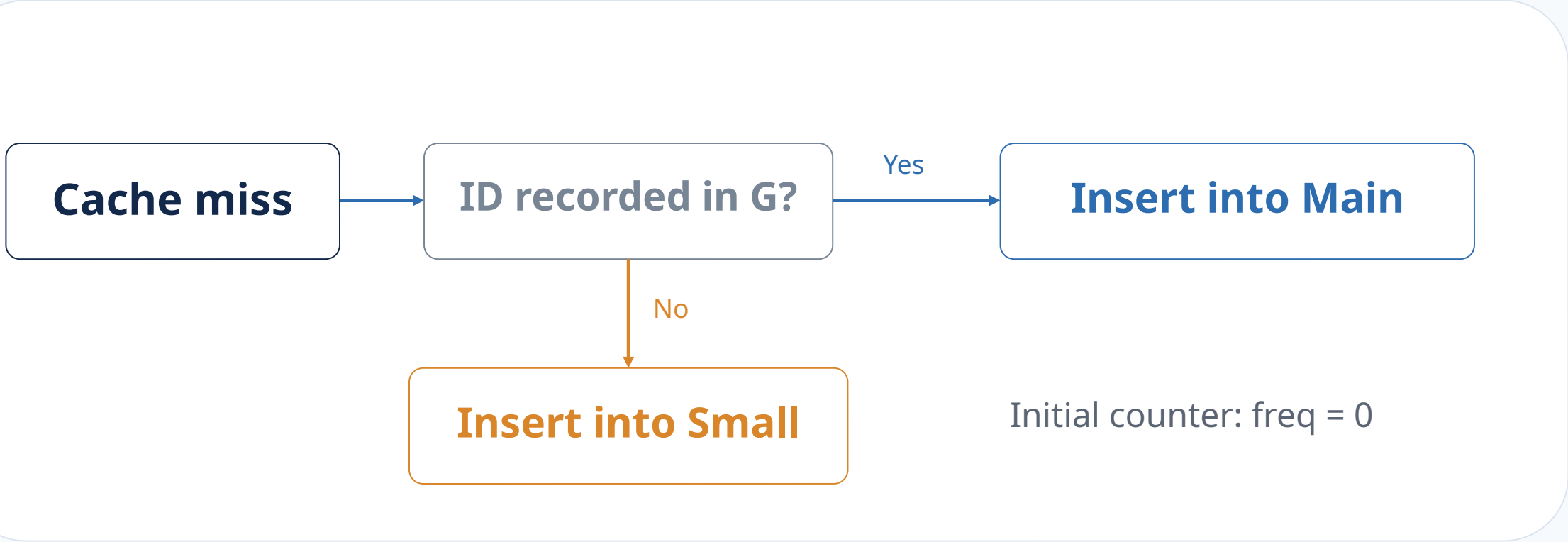
Unlike LRU, S3-FIFO does not move the object after every cache hit.

On hits: update access information, keep the FIFO order unchanged.

Source: Algorithm 1, lines 1–3, Yang et al., SOSP 2023.

Cache Miss: Use Ghost History

If the object is absent from Small and Main, its Ghost membership chooses the destination.

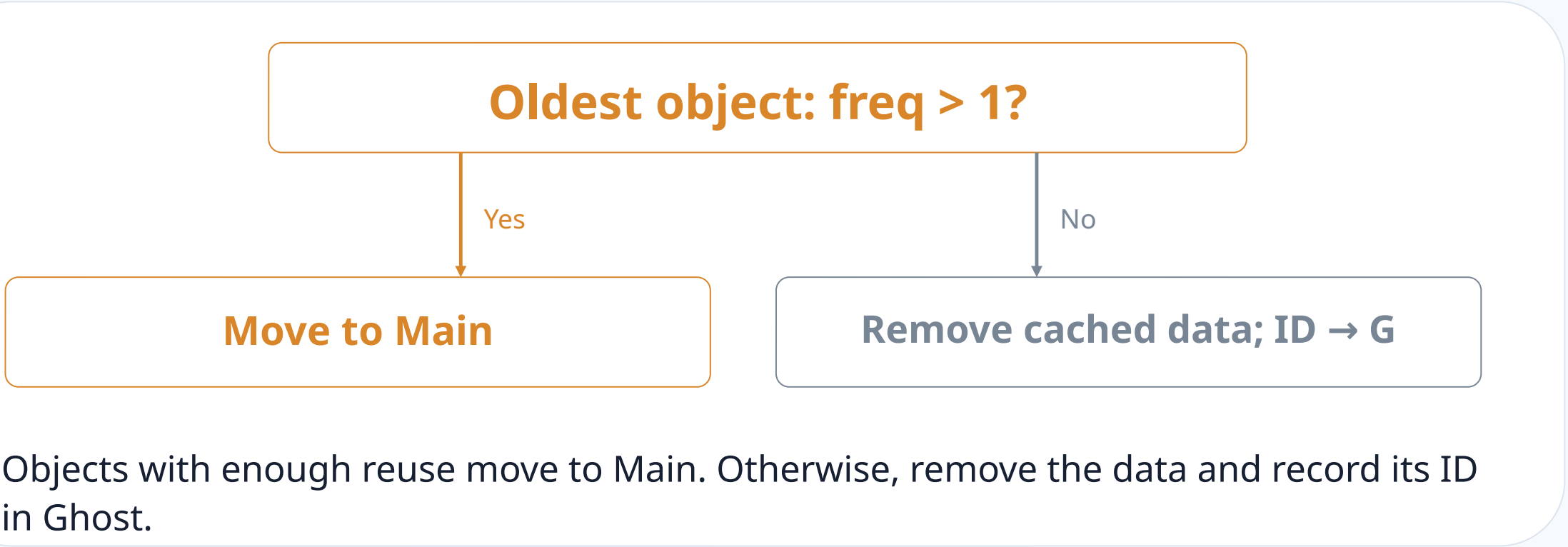


No Ghost history → Small; remembered ID → Main.

Source: Algorithm 1, lines 4–13, Yang et al., SOSP 2023. Simplified admission flowchart.

Small Queue Eviction: Filter New Objects

Inspect the oldest object when this queue needs to release space.



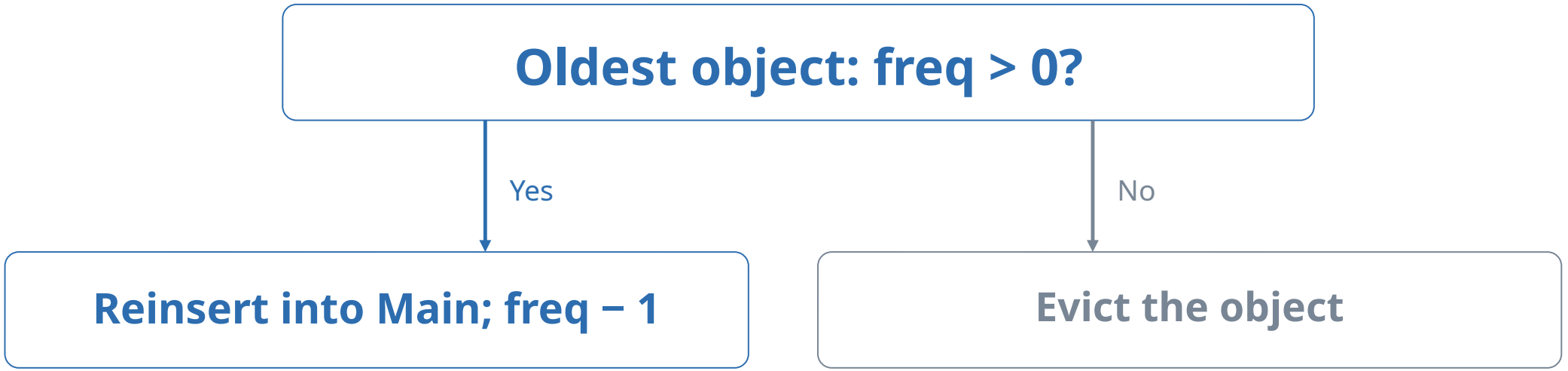
Objects with enough reuse move to Main. Otherwise, remove the data and record its ID in Ghost.

Small filters new objects; Main protects useful objects; Ghost remembers removed IDs.

Source: Algorithm 1, lines 19–30, Yang et al., SOSP 2023. Simplified eviction flowchart.

Main Queue Eviction: A Second Chance

Inspect the oldest object when this queue needs to release space.



An accessed object is reinserted with a smaller counter. If its counter is zero, it is removed.

Small filters new objects; Main protects useful objects; Ghost remembers removed IDs.

Source: Algorithm 1, lines 31–40, Yang et al., SOSP 2023. Simplified eviction flowchart.

PART 3 · MEMBER 3

Evaluation & Conclusion

What do the paper’s experiments show?

Experimental setup → Efficiency → Scalability

Experimental Setup: Real-World Workloads

The researchers evaluated S3-FIFO using a broad collection of cache traces.

6,594
real cache traces

14
datasets / data groups

WORKLOAD CATEGORIES

- **Storage blocks**
- **Key-value data**
- **Web content**

All experimental results are reported by the original paper's authors.

Source: Table 1 and §5.1, Yang et al., SOSP 2023.

What Did the Researchers Compare?

S3-FIFO was compared with FIFO, LRU and other cache replacement methods.

FIFO

Simple insertion-order eviction

LRU

Evict the least recently used object

S3-FIFO

Three static FIFO queues

Other eviction policies were included in the paper’s evaluation as well.

Compare both cache efficiency and the cost of serving requests.

Source: §§5.1–5.3, Yang et al., SOSP 2023.

What Does Miss Ratio Mean?

A cache miss occurs when the requested data is not in the cache.

Miss ratio = cache misses / total requests

A lower miss ratio is better.

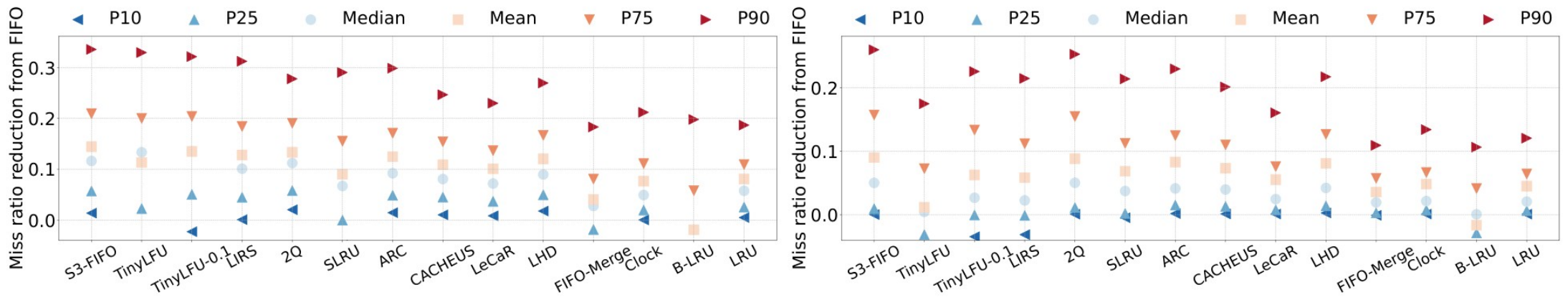
It means more requests can be served directly from the cache.

Miss ratio measures cache efficiency: lower is better.

Source: §5.2, Yang et al., SOSP 2023.

Result 1: Cache Efficiency

Figure 6 compares miss-ratio reduction relative to FIFO, rather than raw miss ratios.



(a) Large cache size, 10% trace footprint

(b) Small cache size, 0.1% trace footprint

Original Figure 6 · Miss-ratio reduction relative to FIFO: a larger reduction is better.

10 / 14

Lowest average miss ratio in 10 of 14 datasets with a relatively large cache.

S3-FIFO performs well across many workloads.

Source: Figure 6 and §5.2, Yang et al., SOSP 2023. The 10/14 dataset result is reported in §5.2 / Figure 7.

What Does Throughput Mean?

Throughput describes how many requests a system can handle in one second.

Throughput = requests handled per second

Higher throughput means more requests can be processed in the same time.

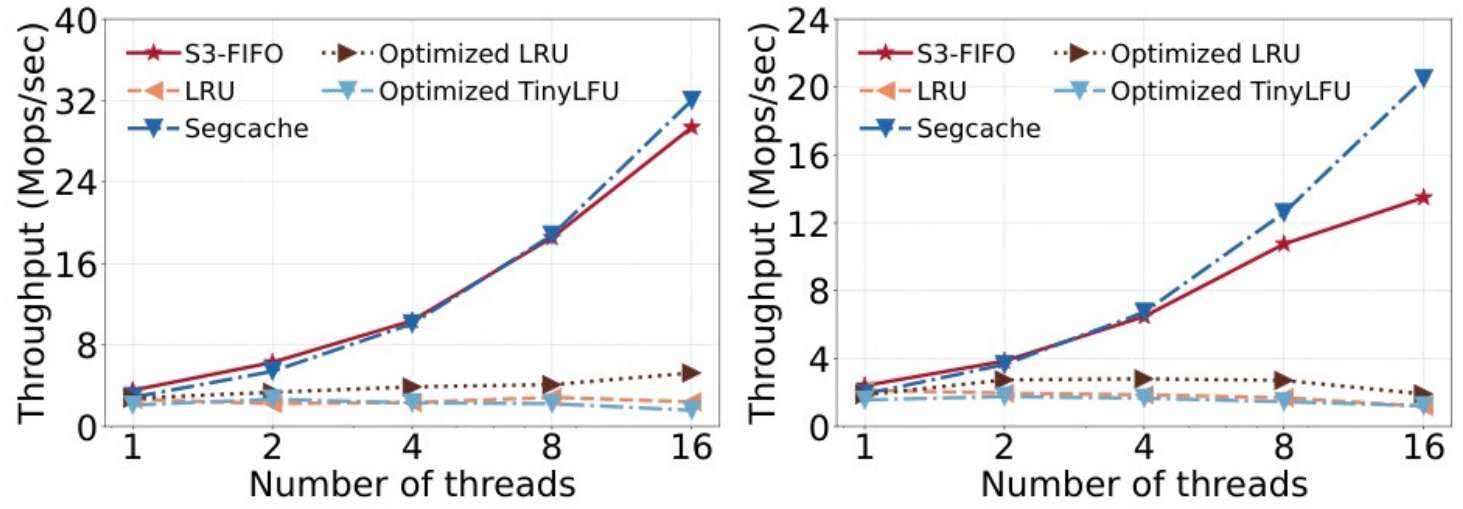
The next figure shows how throughput changes as thread count increases.

Throughput measures processing performance: higher is better.

Source: §5.3, Yang et al., SOSP 2023.

Result 2: Throughput & Scalability

Figure 8 compares throughput as thread count increases.



Original Figure 8 · Synthetic Zipf workload (α = 1.0)

>6×

Compared with optimized LRU at 16 threads.

S3-FIFO scales as the number of threads grows.

Fewer operations on cache hits support better performance under concurrency.

Source: original Figure 8 and §5.3, Yang et al., SOSP 2023. Reported authors' experiments.

A Limitation: The Second Request May Arrive Late

Performance still depends on how users access the data.

Sometimes an object is removed from the Small Queue before its second request arrives.

This can cause an extra cache miss.



Quick demotion is useful, but a late reuse can still cause an additional miss.

Source: §5.2, Yang et al., SOSP 2023.

Final Takeaway

A simple cache algorithm can still deliver strong performance.

Three FIFO queues remove less useful data quickly while keeping useful data for longer.

Lower miss ratios · Better scalability

Thank you for listening.

References

Primary paper, original figures, and the official code and data.

PRIMARY PAPER

Yang, J., Zhang, Y., Qiu, Z., Yue, Y., & Rashmi, K. V. (2023).

FIFO Queues Are All You Need for Cache Eviction. SOSP ’23.

DOI

https://doi.org/10.1145/3600006.3613147

OFFICIAL CODE & DATA

https://github.com/TheSys-lab/sosp23-s3fifo

Figures 1, 5, 6 and 8: Yang et al., SOSP 2023. Mechanism diagrams redrawn from Algorithm 1.